Phase 1 Companion (Code & Reproducibility)

Skander Jebali

October 13, 2025

Purpose. This note explains *what* we implemented in Phase 1, how the files are organized, and how the numerical pieces fit together. It is complementary to the model write-up: here we focus on software structure, data flow, and the precise meaning of each artifact (figures, tables, audit).

1 Big picture (Section 1 of 17)

We built a two-country, rule-based growth–emissions toy model. Countries $i \in \{D, L\}$ produce with Cobb–Douglas technology and follow simple rules for savings, abatement, and one-way migration $L \to D$. A global carbon stock M accumulates emissions; there are no climate damages in Phase 1.

Phase-1 scope. Phase-1 runs use constant active-labor shares (ζ_i). The code also supports time-varying $\zeta_{i,t}$ if provided; we simply do not use that option in Phase-1 experiments.

Micro-notation used in this note. We write ζ for the active-labor share with $L = \zeta N$; $y = Y/N$ (per capita income); $q = Y/L$ (output per active worker); and $\varrho_t = w_{D,t}/w_{L,t}$ (wage ratio driving migration).

The codebase supports:

- deterministic simulation of time paths for (K, N, L, Y, E, M),
- closed-form steady states in efficiency units (Solow),
- four comparative-statics experiments,
- figures (capital dilution; emissions & M_{t+1}/E_t ratio; ambiguity map),
- tables that mirror the comparative-statics summary,

- invariants & light tests, and
- one-command reproducibility with an audit.

2 Repository layout (Section 2 of 17)

The repository `thesis-toy-model/` is organized so that every research artifact has a clear home. Table 1 maps paths to roles and outputs.

Table 1: Repository map (Phase 1).

Path	Role / Type	Key contents & outputs
`model/`	Python package: core logic	Numerical primitives for simulation, steady states, checks. Imported as `model.*`.
`model/params.py`	Parameters & YAML loader	Dataclasses (globals, countries, initial state, migration) with feasibility guards. `load_params("params.yaml")` returns a validated *ModelParams*.
`model/rules.py`	Pure rule functions	Closed-form rules for abatement, intensity, wages, migration, and capital update (no side effects).
`model/solow.py`	Closed-form steady states	Efficiency-unit fixed points; provides $\hat{k}^*$ and $(Y/L)^*$, and the carbon ratio limit helper.
`model/simulate.py`	Deterministic simulator	Period timing and state updates; returns paths for $(Y/L)_i$, y_i, E_t, M_t, and recorded M_{t+1}/E_t.
`model/checks.py`	Invariants & asserts	Positivity, budget feasibility, migration caps, BGP ratio condition; called during simulation.
`experiments/`	Experiment engine	Baseline vs. shock orchestration.
`experiments/shocks.py`	Shock helpers	Overlays for `nD_up`, `nL_up`, `abatement_up`, `ease_migration`, `ease_migration_strong`.
`experiments/runner.py`	Main orchestrator	`run_all(...)` returns nested results for plots/tables (CSV written by `viz/tables.py`).
`viz/`	Plotting & tables	Publication figures and CS tables.

continued on next page

Path	Role / Type	Key contents & outputs
`viz/plots.py`	Matplotlib figures	`capital_dilution_plot(...)`, `emissions_and_ratio_plot(...)`, `ambiguity_map(...)`; outputs to `figures/`.
`viz/diagnostics.py`	Diagnostic plots	`migration_diagnostics_plot(...)`: 2×2 panel for migration shocks (paths of m_t, ϱ_t, N_L, L_D); saved for `ease_migration` and `ease_migration_strong`.
`viz/tables.py`	Pandas CS tables	Builds comparative-statics table; saved as `results/cs_table.csv`.
`tests/`	Pytest unit tests	Theory checks (Solow, elasticity threshold, carbon ratio limit, BGP growth, budget feasibility) and simulation sign tests.
`figures/`	Output folder (created)	All `.png` figures with stable names encoding shocks.
`results/`	Output folder (created)	`cs_table.csv` and `run_summary.txt` (timestamped notes).
`scripts/`	Utility scripts	`scripts/sweep_ease_migration.py`: grid over μ and $\Delta\tau_H$; writes `results/ease_migration_sweep.csv`. Run from repo root with `python-mscripts.sweep_ease_migration` (Windows: `setPYTHONPATH=%CD%`).
`params.yaml`	Parameter file	Human-readable configuration: globals, per-country blocks, migration. Loaded by `model/params.py`.
`make_all_figures.py`	One-command harness	End-to-end generator: runs experiments, figures, tables, checks; writes `results/run_summary.txt`.
`audit_phase1.py`	Post-run audit	Verifies outputs exist and identities hold; prints concise checklist.
`README.md`	Top-level guide	Quick-start instructions and reproduction pointers.

Naming discipline. Filenames encode (i) the category and (ii) the shock (e.g. `fig_emissions_ratio_nD_up.png`). This keeps manuscript cross-references stable. We also denote the active-labor share by ζ (so $L = \zeta N$). In YAML/code the key is `zeta`. *Older drafts used ς; we no longer do.*

3 Parameters and structure (Section 3 of 17)

Parameters are grouped to keep YAML clean and to separate universal vs. country-specific quantities.

Global (shared across countries). $\alpha \in (0,1)$ (capital share); $\delta \in (0,1)$ (depreciation); $\phi_M \in (0,1)$ (carbon persistence); g_A (labor-augmenting productivity growth: $A_{i,t+1} = (1+g_A)A_{i,t}$, with $1+g_A > 0$); $q^\star > 0$ (scaling constant in the intensity function; $q = Y/L$); $\eta \geq 0$ (productivity–intensity elasticity).

Per-country $i \in \{D, L\}$. Savings $s_i \in (0,1)$; net fertility $f_i > -1$; baseline emissions intensity $\varepsilon_i > 0$; abatement cost weight $\kappa_i > 0$; abatement responsiveness $\xi_i \geq 0$; abatement cap $\bar{\theta}_i \in (0,1]$; *active-labor share $\zeta_i \in (0,1]$. Phase 1 uses constant ζ_i (scalars) for reproducibility; the code also supports time-varying $\zeta_{i,t}$, which we do not use in Phase 1.*

Migration. Responsiveness $\mu > 0$ *(units: 1/period)*; headcount policy wedge $\tau_H \geq 0$ *(persons/period)*; feasibility cap $\bar{m} \in (0,1]$ *(share of $N_{L,t}$, persons scaling).* We use a *strict-slack* calibration buffer `mig_slack`>0 implementing

$$\bar{m} \;\leq\; \min\{\,1,\; 1 + f_L - \texttt{mig_slack}\,\},$$

so the $(1+f_L)N_{L,t}$ branch of the cap is slack in baseline runs.
Note. `mig_slack` = strict-slack buffer for migration feasibility; unrelated to emissions intensity ε. We validate $\tau_H \geq 0$ on load; shock overlays that would push $\tau_H < 0$ are *clipped to zero*, so "strong" easing reduces τ_H only down to zero.

Feasibility guards (checked on load and during simulation). Budget feasibility $s_i + \kappa_i \theta_{i,t}^2 \leq 1$ *by construction* because $\theta = \min\{\bar{\theta}, \xi y, \sqrt{(1-s)/\kappa}\}$ is capped; optionally enforce $s_i + \kappa_i \bar{\theta}_i^2 \leq 1$ so $C \geq 0$ even when $\theta = \bar{\theta}$. Wages and outputs are strictly positive under Cobb–Douglas with $K > 0$, $A > 0$, $L > 0$; migration is clipped (persons) at $\min\{\bar{m}N_{L,t}, (1+f_L)N_{L,t}\}$. We impose a strict-slack calibration $\bar{m} \leq \min\{1,\, 1 + f_L -$ mig_slack$\}$ with `mig_slack`> 0 *(`mig_slack` = strict-slack buffer for migration feasibility; unrelated to emissions intensity ε)*, so the $(1+f_L)$ branch of the cap is slack in baseline and $N_{L,t} > 0$ holds for all t.

Units cheat-sheet.

- μ: 1/period (migration responsiveness); τ_H: persons/period (headcount wedge); $\bar{m}$: *share* (cap applied as $\bar{m}\, N_L$ persons).
- ε: emissions per good (so $E = \varepsilon(q)\,(1-\theta)\,Y$ is in physical emissions units).
- E, M: physical units (e.g., $GtCO_2e$).

Note on parameter values (with rationale)

Below is what the current `params.yaml` effectively encodes (as reported by the code).

Globals.

`alpha` = 0.33	Standard capital share in macro calibrations.
`delta` = 0.06	Annual depreciation (∼6%).
`phi_M` = 0.90	Carbon persistence; typical order of magnitude.
`g_A` = 0.0	Stationary technology in Phase 1 to keep per-capita levels comparable.
`q_star` = 1.0	Scaling for productivity-dependent intensity ($q = Y/L$).
`eta` = 0.30	Small positive elasticity for the "higher-productivity ⇒ higher intensity" channel; conservative relative to earlier 0.5.

Country D (Developed).

`s` = 0.26	Slightly higher saving rate than L.
`f` = 0.003	Very low net natural increase (stylized fact for rich countries).
`epsilon` = 0.45	Lower emissions intensity than L.
`kappa` = 0.20	Abatement cost weight.
`xi` = 0.45	Stronger abatement responsiveness to income per capita.
`theta_bar` = 0.85	Higher abatement cap than L.
`zeta` = 0.92	Lower active-labor share (aging proxy).

Country L (Developing).

`s` = **0.20**	Lower saving rate than D.
`f` = **0.015**	Positive net natural increase.
`epsilon` = **0.70**	Higher emissions intensity.
`kappa` = **0.20**	Abatement cost weight (matched to D for clarity).
`xi` = **0.25**	Weaker abatement responsiveness than D.
`theta_bar` = **0.65**	Lower abatement cap than D.
`zeta` = **0.98**	Near full active-labor share.

Migration.

`mu` = **0.06**, `tau_H` = **0.08**, `m_bar` = **0.10**, `mig_slack` = **0.02**

Moderate responsiveness, modest policy wedge, a 10% cap (headcount), and a strict-slack buffer (parameter `mig_slack`; not the emissions-intensity ε) so the $(1 + f_L)$ branch is slack.

Initial state.

`K_D` $= 1.0$, `K_L` $= 1.0$

`N_D` $= 1.0$, `N_L` $= 1.0$

`A0_D` $= 1.0$, `A0_L` $= 1.0$, `M0` $= 0.0$

Notation. We use math subscripts $A_{i,0}$ in formulas, while the YAML keys are `A0_D`, `A0_L`.

Simulation.

`T` = **200**

Horizon used in reproducibility runs and figures.

Scenarios.

- **`nD_up`:** small bump in f_D (higher n_D^{act}) to illustrate capital dilution in D.
- **`nL_up`:** small bump in f_L (higher n_L^{act}) to illustrate capital dilution in L.
- **`abatement_up`:** raises (ξ, θ_bar) in both countries (D: $0.60, 0.90$; L: $0.40, 0.80$).
- **`ease_migration`:** higher μ and lower τ_H to relax migration frictions.
- **`aging_in_D`:** lowers ζ_D (e.g., to 0.90) for that scenario.

4 Core rule functions (Section 4 of 17)

All rule functions are pure maps: inputs → outputs with no side effects.

- **Abatement:** $\theta(y) = \min\{\bar{\theta},\ \xi y,\ \sqrt{(1-s)/\kappa}\} \in [0,1]$.
- **Emissions intensity:** $\varepsilon(q) = \varepsilon\,(q/q^\star)^\eta$ with $q = Y/L$.
- **Emissions:** $E = \varepsilon(q)\,(1-\theta(y))\,Y$, with $\varepsilon(q) = \varepsilon\,(q/q^\star)^\eta$ and $\theta(y) = \min\{\bar{\theta}, \xi y, \sqrt{(1-s)/\kappa}\}$.
- **Wage per active worker:** $w_{i,t} = (1-\alpha)\,Y_{i,t}/L_{i,t}$ with $L_{i,t} = \zeta_i N_{i,t}$.
- **Migration (one way $L \to D$):** Let $\varrho_t \equiv \frac{w_{D,t}}{w_{L,t}}$. Then

$$m_t = \min\Big\{\,[\,\mu\,\varrho_t\,N_{L,t} - \tau_H\,]_+\,,\ \bar{m}\,N_{L,t},\ (1+f_L)N_{L,t}\Big\}\,.$$

  Note. Under our strict-slack calibration ($\bar{m} \le \min\{1,\ 1+f_L - \text{mig_slack}\}$), the $(1+f_L)N_{L,t}$ branch is slack.
  Rationale. Using the wage *ratio* ϱ_t avoids mechanical scaling from common TFP growth g_A.

- **Capital update:** $K_{i,t+1} = (1-\delta)K_{i,t} + s_i Y_{i,t}$.

5 Closed-form steady states (Section 5 of 17)

Define effective capital $\hat{k}_{i,t} = K_{i,t}/(A_{i,t}L_{i,t})$. With $A_{i,t+1} = (1+g_A)A_{i,t}$ and $L_{i,t+1} = (1+n_i^{\text{act}})L_{i,t}$,

$$\hat{k}^* = \left(\frac{s_i}{\delta + g_A + n_i^{\text{act}} + g_A\,n_i^{\text{act}}}\right)^{\frac{1}{1-\alpha}}, \qquad \left(\frac{Y}{L}\right)^* = A_{i,t}\,(\hat{k}^*)^\alpha.$$

Per-active-worker to per-capita mapping. Since $y = \dfrac{Y}{N} = \zeta\,\dfrac{Y}{L}$, at steady state

$$y^* = \zeta_i\,A_{i,t}\,(\hat{k}^*)^\alpha.$$

Finiteness condition. $\hat{k}^*$ is finite iff

$$n_i^{\text{act}} \;>\; -\frac{\delta + g_A}{1+g_A}.$$

The dilution comparative static is standard: $\frac{\partial \hat{k}^*}{\partial n_i^{\text{act}}} < 0$ and therefore $\frac{\partial\Big((Y/L)_i^*/A_{i,t}\Big)}{\partial n_i^{\text{act}}} < 0$.

Interpretation. Steady states are in efficiency units. Levels grow with $A_{i,t}$. We use the closed form to benchmark simulated transitions and to verify signs under the demographic shocks.

6 Simulator (Section 6 of 17)

Each period executes the following *timing*, which is fixed across experiments:

1. **Production:** $Y_{i,t} = K_{i,t}^{\alpha}(A_{i,t}L_{i,t})^{1-\alpha}$.
2. **Prices:** $w_{i,t} = (1-\alpha)Y_{i,t}/L_{i,t}$, $r_{i,t} = \alpha Y_{i,t}/K_{i,t}$.
3. **Rules:** compute $y_{i,t} = Y_{i,t}/N_{i,t}$ and $q_{i,t} = Y_{i,t}/L_{i,t}$; then set $\theta_{i,t} = \theta_i(y_{i,t})$, $\varepsilon_{i,t} = \varepsilon_i\left(\frac{q_{i,t}}{q^\star}\right)^{\eta}$, and migration m_t.
4. **Feasibility:** enforce the abatement budget cap and the migration cap; assert positivity.
5. **Updates:** $K_{i,t+1} = (1-\delta)K_{i,t} + s_iY_{i,t}$; populations $N_{D,t+1} = (1+f_D)N_{D,t} + m_t$, $N_{L,t+1} = (1+f_L)N_{L,t} - m_t$; active labor $L_{i,t+1} = \zeta_{i,t+1}\,N_{i,t+1}$ if a schedule $\zeta_{i,t}$ is provided, else $L_{i,t+1} = \zeta_i N_{i,t+1}$ *(Phase 1: ζ_i is constant).* Compute world emissions $E_t = \sum_i \varepsilon_{i,t}\,(1-\theta_{i,t})\,Y_{i,t}$; then update the carbon stock $M_{t+1} = \phi_M M_t + E_t$; technology $A_{i,t+1} = (1+g_A)A_{i,t}$.

   Units. Emissions E and the stock M are in physical units; ε is "emissions per good."

The simulator returns arrays for $(Y/L)_i$, y_i, world E_t, M_t, and the recorded ratio M_{t+1}/E_t (we do not separately store M_t/E_t). Figures use this recorded ratio (useful when levels of M do not converge under $g_E > 0$).

7 Experiments (Section 7 of 17)

We run a baseline and four comparative-statics shocks, plus a stronger migration variant:

(a) **`nD_up`**: higher labor growth in D (implemented via a small bump in f_D); capital dilution lowers $(Y/L)_D$.

(b) **`nL_up`**: higher labor growth in L; dilution lowers $(Y/L)_L$.

(c) **`abatement_up`**: stronger mitigation (increase ξ_i and/or $\bar{\theta}_i$); world E falls.

(d) **`ease_migration`**: policy easing in the rule $m_t = \min\{\,[\mu\,\varrho_t N_{L,t} - \tau_H]_+,\ \bar{m}N_{L,t},\ (1+f_L)N_{L,t}\}$.

We also include **`ease_migration_strong`** (μ higher, τ_H lower) as a labeled *variant* to illustrate when the textbook dilution sign reappears under persistent tail flows. The orchestrator `experiments/runner.py` collects baseline vs. shock paths into a single nested object used by plots and tables.

Note: Endogenous rule vs. exogenous n. Easing ($\mu \uparrow$, $\tau_H \downarrow$) front-loads flows; as N_L shrinks and ϱ_t narrows, $\mu\, \varrho_t N_L - \tau_H$ can decline and tail migration fall below baseline, so $(Y/L)_D$ need not be lower at the horizon. The `ease_migration_strong` variant shows that when tail flows remain elevated, the textbook dilution sign re-appears.

Auxiliary scenario (not part of the four CS shocks). For convenience, the repository also includes a scenario `aging_in_D` that sets the active-labor share ζ_D to a lower constant. This is *not* one of the four headline comparative-statics experiments and is excluded from the CS figures and the comparative-statics CSV (`results/cs_table.csv`); it is provided only as an extra illustration for sensitivity to demographics.

8 Visualization & tables (Section 8 of 17)

We generate the following figures and CSV tables:

Figures.

- **Capital dilution panels** for `nD_up` and `nL_up`: left pane $(Y/L)_D$, right pane $(Y/L)_L$, baseline (solid) vs. shock (dashed).
- **Emissions & ratio panels** (all shocks, including the labeled variant): left pane world E_t, right pane M_{t+1}/E_t; shows level shifts and the long-run ratio convergence. We compute the recorded ratio as M_{t+1}/E_t from stored arrays (shift M forward one step and divide by current E). *Limit mapping:* if $E_{t+1}/E_t \to 1 + g_E$ with $(1 + g_E) > \phi_M$,

$$\frac{M_t}{E_t} \to \frac{1}{(1+g_E) - \phi_M} \quad \Longrightarrow \quad \frac{M_{t+1}}{E_t} \to \frac{1+g_E}{(1+g_E) - \phi_M}.$$

- **Ambiguity map** (parameter η annotated): the threshold $\xi y = \frac{1}{2}$ (holding q fixed) partitions regions where $\partial \ln e / \partial \ln y$ is positive vs. negative *only when the abatement rule is interior* ($\theta = \xi y$). **If a cap binds** ($\theta = \bar{\theta}$ or $\theta = \sqrt{(1-s)/\kappa}$), then $e \propto q^\eta y$ and the elasticities are 1 in y and η in q.
- **Migration diagnostics** (for `ease_migration` and `ease_migration_strong`): 2×2 panel with m_t, ϱ_t, $N_{L,t}$, $L_{D,t}$. Makes the front-loading vs. tail-easing mechanism visible. Filenames: `fig_migration_diagnostics_ease_migration.png`, `fig_migration_diagnostics_ease_migration_strong.png`.

All figures are written to `figures/fig_*.png` with stable names (shock encoded).

Comparative-statics CSV (results/cs_table.csv). Columns written by the code:

- `Shock`
- `(Y/L)_D`, `(Y/L)_D % change`
- `(Y/L)_L`, `(Y/L)_L % change`
- `World E`, `World E % change`
- `M level (if stationary)`, `M % change`
- `M_{t+1}/E_t (else)`, `(M_{t+1}/E_t) % change`

If both baseline and shock appear stationary (tail $g_E \approx 0$ in each), the table compares M levels. Otherwise it compares tail averages of the recorded ratio M_{t+1}/E_t.

M-column note (conventions). In stationary runs ($g_E = 0$), arrows for M mirror E and a level $M^* = E^*/(1-\phi_M)$ exists. Along growth paths ($g_E > 0$), the CSV reports the *recorded ratio* M_{t+1}/E_t columns (arrows and $\%\Delta$). (Theory uses $M_t/E_t \to 1/((1+g_E)-\phi_M)$; the recorded ratio converges to $(1+g_E)/((1+g_E)-\phi_M)$.) To diagnose stationarity, we compute a tail growth estimate $\hat{g}_E = \frac{1}{K}\sum_{t=T-K+1}^{T}\Big(\ln E_t - \ln E_{t-1}\Big)$ with $K = 20$; we treat runs as stationary when $|\hat{g}_E| < 10^{-3}$.
In non-stationary cases, we therefore report `M_{t+1}/E_t` (not M_t/E_t) in the CSV. We set the tail window and threshold as module-level constants in `viz/tables.py`: `TAIL_WINDOW_K=20`, `STATIONARY_ABS_GROWTH_THRESHOLD=1e-3`.

Reproduction (one command). From the repository root:

```
python make_all_figures.py
```

This regenerates *all* figures and tables, re-runs baseline checks, and writes a run log to `results/run_summary.txt`.

9 Sanity checks & invariants (Section 9 of 17)

We enforce numerical and logical consistency at two levels: (i) *on-the-fly* checks during simulation, and (ii) *post hoc* checks on stored paths. Table 2 summarizes what is enforced and why it matters.

Table 2: Core invariants and their purpose.

Object	Invariant	Why / Notes
Output & inputs	$K_{i,t} > 0,\ A_{i,t} > 0,\ L_{i,t} > 0 \Rightarrow Y_{i,t} > 0$	Cobb–Douglas monotonicity ensures positivity; prevents division by zero.
Wages	$w_{i,t} = (1-\alpha)Y_{i,t}/L_{i,t} > 0$	Migration rule requires $w_{L,t} > 0$ for the relative gap.
Budget feasibility	$C_{i,t} = (1 - s_i - \kappa_i\theta_{i,t}^2)Y_{i,t} \geq 0$	Guaranteed by $\theta = \min\{\bar{\theta}, \xi y, \sqrt{(1-s)/\kappa}\}$.
Migration cap	$0 \leq m_t \leq \min\{\bar{m}N_{L,t}, (1+f_L)N_{L,t}\}$	Caps flows in *persons*; avoids negative $N_{L,t+1}$.
Population positivity	$N_{i,t+1} > 0$ for all t	With $\bar{m} \leq \min\{1, 1 + f_L - \text{mig_slack}\}$.
BGP ratio condition	If $g_E > 0$, require $(1+g_E) > \phi_M$	Ensures $M_t/E_t \to 1/((1+g_E) - \phi_M)$ is well-defined.
Solow finiteness	$n_i^{\text{act}} > -\dfrac{\delta + g_A}{1+g_A}$	Ensures the closed-form $\hat{k}^*$ is finite and the fixed point exists.

Where checks live. Runtime asserts live in `model/checks.py` and are called from `simulate.py`. Advisory warnings (e.g., BGP condition) do not stop execution but are recorded in the log.

What happens on failure. Assertions raise a Python `AssertionError` (or `ValueError`) with a concise message indicating which invariant was violated and at which time step / country.

10 Light testing (Section 10 of 17)

Phase-1 tests are intentionally "unit-size," fast, and theory-targeted.

Theoretical identities (tests/test_theory.py; mirrors model Appendix C.1–C.5).

- **C.1 (Solow monotonicity):** verifies $\hat{k}^* > 0$ and the capital-dilution comparative static $\partial\hat{k}^*/\partial n < 0$ numerically.
- **C.2 (Elasticity threshold):** for interior abatement $\theta = \xi y$, verifies that $\partial \ln e/\partial \ln y$ changes sign at $\xi y = \frac{1}{2}$ (holding q fixed).

- **C.3 (Carbon ratio limit):** with a synthetic $E_{t+1}/E_t \to 1 + g_E$, checks $M_t/E_t \to 1/((1 + g_E) - \phi_M)$.
- **C.4 (Emissions growth on a BGP):** with $g_A > 0$, constant ζ and a binding abatement share, construct a synthetic path and verify $\hat{g}_E \approx (\eta + 1)g_A + n^{\text{act}}_{\text{world}}$.
- **C.5 (Budget feasibility across branches):** sample values of y that make θ interior and capped; check $C = (1 - s - \kappa\theta^2)Y \geq 0$ always holds.

Simulation checks (tests/test_simulation.py).

- **Smoke & invariants:** loads `params.yaml`, simulates a short horizon, asserts basic positivity and runs the invariant suite.
- **Sign checks:** under `nD_up` (`nL_up`), $(Y/L)_D$ (resp. $(Y/L)_L$) falls relative to baseline; under `abatement_up`, world E_t falls.

How to run.

```
pytest -q tests
```

A clean run shows all tests passing; failures print a minimal counterexample and a hint.

11 Reproducibility harness (Section 11 of 17)

The script `make_all_figures.py` is the *single-command* generator for all Phase-1 artifacts.

What it does, in order.

1. Load `params.yaml` with feasibility guards.
2. Run the baseline, four shocks (`nD_up`, `nL_up`, `abatement_up`, `ease_migration`) *and* the labeled variant `ease_migration_strong` for horizon T (default 200).
3. Produce all **figures** in `figures/` and **tables** in `results/`.
4. Re-run **sanity checks** on the baseline simulation.
5. Write `results/run_summary.txt` (timestamp, paths, counts).

How to run.

```
cd <repo-root>  &  python make_all_figures.py
```

The console prints a clear `=== Reproducibility run finished successfully ===` when done.

12 Audit script (Section 12 of 17)

`audit_phase1.py` verifies *outputs exist* and *quantitative identities* hold, then prints a compact checklist. Typical output:

```
=== Phase 1 Audit (single-spec) ===
[PASS] A1.D Solow fixed point (rel err 3.9%, n_hat=0.0021)
[PASS] A1.L Solow fixed point (rel err 0.8%, n_hat=0.0148)
[PASS] A2 M/E condition: 1+g_E=1.012 vs phi_M=0.900
[PASS] B Invariants (positivity & budget)
[PASS] C nD_up: (Y/L)_D falls
[PASS] C nL_up: (Y/L)_L falls
[PASS] C abatement_up: World E falls
[PASS] D1 Figures generated (12 png files found)
[PASS] D2 CS table (results/cs_table.csv)
[PASS] D3 Run summary (results/run_summary.txt)

=== Audit Summary ===
Passed: 10
Failed: 0
```

Scope. The audit runs quick, short-horizon simulations (for Solow and sign checks) and inspects artifacts produced by the reproducibility run; it does not regenerate the full figure/table set.

13 How to (re)run, export, and share (Section 13 of 17)

Minimal re-run.

```
python make_all_figures.py  &  python audit_phase1.py
```

Sensitivity sweep (optional). `python -m scripts.sweep_ease_migration` writes `results/ease_migration_sweep.csv` summarizing tail migration changes and signs over a small grid in μ and $\Delta\tau_H$.

What to send your advisor.

- **Figures**: `figures/*.png` (stable file names).
- **Tables**: `results/cs_table.csv`.

- **Run log**: `results/run_summary.txt`.
- **Params**: `params.yaml` (so results are interpretable).

Packaging suggestion. Create a lightweight bundle that excludes `__pycache__/`:

```
zip -r phase1_artifact_pack.zip figures results params.yaml README.md
↪ requirements.txt environment.yaml
```

14 Environment, determinism, and provenance (Section 14 of 17)

Python & deps. Python $\geq$ 3.10; `numpy`, `pandas`, `matplotlib`, `pytest`, `PyYAML`. Non-interactive backend set via `MPLBACKEND=Agg` to avoid display coupling.

Re-building the env (recommended).

- **Conda (if used):** `conda env export > environment.yaml` to capture the working set; restore via `conda env create -f environment.yaml`.
- **Pip:** `python -m pip freeze > requirements.txt`; restore via `python -m pip install -r requirements.txt`.

Install PyYAML explicitly (if not present).

- Pip: `python -m pip install pyyaml`
- Conda: `conda install -y pyyaml`

Determinism. No PRNG is used in Phase 1; dynamics are deterministic. Figures depend only on `params.yaml`. Matplotlib style is default to avoid system-specific fonts.

Provenance. The runner writes a timestamped `run_summary.txt`; commit the repo before/after a run to pin the exact code state:

```
git add -A; git commit -m "Phase1 reproducibility run";
```

15 File naming and data schemas (Section 15 of 17)

Figures. Names follow:

```
fig_<category>_<shock>.png
```

Examples: `fig_capital_dilution_nD_up.png`; `fig_emissions_ratio_ease_migration.png`; `fig_ambiguity_eta-0.30.png`. We standardize the ambiguity-map suffix as `eta-<two-decimals>` (hyphen + two decimals).

Comparative-statics CSV (results/cs_table.csv). Columns written by the code:

- `Shock`
- `(Y/L)_D`, `(Y/L)_D % change`
- `(Y/L)_L`, `(Y/L)_L % change`
- `World E`, `World E % change`
- `M level (if stationary)`, `M % change`
- `M_{t+1}/E_t (else)`, `(M_{t+1}/E_t) % change`

Arrow glyphs: "↑", "↓", "∅" (no meaningful change), "(n/a)" (not applicable for that branch). The "M" vs "M_{t+1}/E_t" choice follows the stationarity rule described in the Visualization section.

Run summary (results/run_summary.txt). Plain text: timestamp, repo path, counts of figures/tables, and a note that sanity checks passed.

16 How to extend safely (Section 16 of 17)

Add a new shock.

1. Create a helper in `experiments/shocks.py` that returns a modified parameter overlay.
2. Register it in `experiments/runner.py`.
3. Add a figure/table stanza if applicable in `viz/plots.py` and `viz/tables.py`.
4. Add at least one sign test in `tests/` mirroring the intended effect.

Add a new figure.

1. Implement a pure function in `viz/plots.py` that takes model outputs and `outdir`.
2. Use consistent labeling (axes, legend) and follow the naming scheme.
3. Call it from `make_all_figures.py`.

Add a new rule or feature.

1. Implement the pure function in `model/rules.py`.
2. Thread the option through `params.yaml` → `params.py` → `simulate.py`.
3. Update invariants in `checks.py` if new caps/feasibility constraints arise.
4. Add a minimal unit test in `tests/`.
5. **Time-varying active-labor shares:** promote ζ_i from scalars to paths by threading a per-period schedule through `params.yaml` → `params.py` and reading it in `simulate.py`; maintain scalar fallback (broadcast over t) for backward compatibility.

Style discipline. Keep rule functions pure; avoid hidden state; pass primitives explicitly; add docstrings with parameter domains and units.

17 Limitations and next steps (Section 17 of 17)

Phase 1 deliberately omits several elements that are natural targets for Phase 2+:

- **No climate damages:** M does not feed back into Y; adding a damages block would couple mitigation and growth margins.
- **No optimization:** countries follow rules; a Ramsey or planner problem would provide welfare metrics and optimal policies.
- **No cohort demography:** the active-labor toggle is a proxy; overlapping generations or age-structured dynamics would sharpen migration/aging trade-offs.
- **Calibration & data:** Phase 1 uses stylized parameters; empirical calibration and sensitivity analysis are natural extensions.
- **Uncertainty:** all paths are deterministic; stochastic productivity or policy shocks would require seeds and distributional outputs.

Phase 2 sketch. (i) introduce marginal damages and an SCC-style mapping; (ii) calibrate $(\varepsilon, \kappa, \xi, \bar{\theta})$ to match historical intensities and MAC curves; (iii) implement welfare accounting; (iv) expand the experiment engine to counterfactual policy packages with sensitivity bands.